

Guessing the Next Pitch: A Machine Learning Approach to Fastball vs. Offspeed Prediction

Luke Kurta, Nathanael Choi, and Francesco Noble

Abstract

Our objective with this project had two steps: first, to use machine learning models and pitch-level context MLB data to assess game context and predict whether a pitcher will throw a fastball or offspeed pitch in a given moment, and secondly to analyze the performance of differently built models to make conclusions about pitchers' habits. To build our models, we used publicly available data scraped from MLB's Statcast database, Baseball Savant, which included hundreds of thousands of pitches, utilizing the informative features embedded in the data and engineering some of our own, such as a count bucket and score differential.

The model type we used was XGBoost, and we created two separate versions, one that created a separate model for each pitcher and one that operated as a global model, not taking pitcher identity into account. The model that incorporates identity tracking was accurate on 61.6% of its predictions, outperforming the global model's 57.7% accuracy. Through analysis of feature importance, we discovered that each model relied on different features, likely due to the difference in approaches, looking at pitcher-specific trends versus league-wide trends. We also looked closely at the first pitch in an at-bat, discovering that pitchers often diverge from their pitch tendencies in these scenarios. We concluded that identity confounding, which is often seen as a modeling flaw, can help reveal individualized habits and strategic variation in real game situations.

Introduction_ exploratory data analysis.ipynb

Baseball is a game of anticipation. Players have to react in split seconds to their opponents' decisions, and a key part of this is a batter's ability to guess what kind of pitch is coming next. Pitchers have a variety of pitches like fastballs, sliders, curveballs, and changeups, each with its own speed, movement, and spin. Even players and coaches at the highest level find predicting the next pitch challenging, but if they're able to at least guess the type of pitch, batters can seriously increase their chances of getting a base hit.

Predictive models for baseball pitching aren't anything new. For example, Dhar (2022) studied general patterns in pitch sequences to predict upcoming pitches, while Lee (2022) used deep neural networks to predict both the type and location of pitches, achieving better accuracy than simpler models. These studies show how advanced the techniques are becoming to handle the unpredictable nature of baseball.

Predicting whether a pitcher will throw a fastball or an offspeed pitch given the game situation is important but tricky: fastballs and offspeed pitches differ not just in speed but also in how they're used strategically, depending on factors like the count (balls and strikes), baserunners, score difference, inning, and whether the batter and pitcher are right or left-handed. We classified fastballs as either a 4-seam fastball, cutter, sinker, splitter, or forkball, whereas offspeed pitches constitute the rest of the pitches. To be clear, fastballs are pitches that are fast (usually above 90 miles per hour at the MLB level) and, for the most part, direct or straight. Offspeed pitches are slower and demonstrate sideward and/or downward motion due to spin, usually meant for the batter to lose his timing and perhaps 'chase' the pitch.

To build our pitch prediction model, we used publicly available pitch-level data from Major League Baseball's Statcast system, accessed via the pybaseball Python library. We collected data from April to September 2024, which covers almost the entire MLB regular season. After downloading the raw dataset, we selected only the features most relevant to pitch context and decision-making: `player_name`, `p_throws`, `stand`, `inning`, `outs_when_up`, `on_1b`, `on_2b`, `on_3b`, `bat_score`, `fld_score`, `pitch_type`, `balls`, and `strikes`. We exported this filtered dataset to CSV.

To make sure we had enough data for individual pitcher tendencies, we filtered the dataset even further to include only pitchers who appeared in at least 300 rows, equivalent to throwing 300 pitches, since each row represents a single pitch event. We did this by grouping the data by pitcher name and counting their occurrences. This is slightly different from the methodology we used when constructing our model (see **Data Collection**), but since it accesses the same data and essentially accomplishes the same thing, for our purposes, it yields the same numbers (reducing the dataset from about 850 pitchers to nearly 600).

Exploratory Data Analysis

To better understand pitch tendencies and guide our modeling, we began with exploratory data analysis (EDA). First, we wanted to check what the most common pitch type was in general, and to no one's surprise, as shown in the figure below, fastballs are by far the most common pitch thrown across all pitchers.

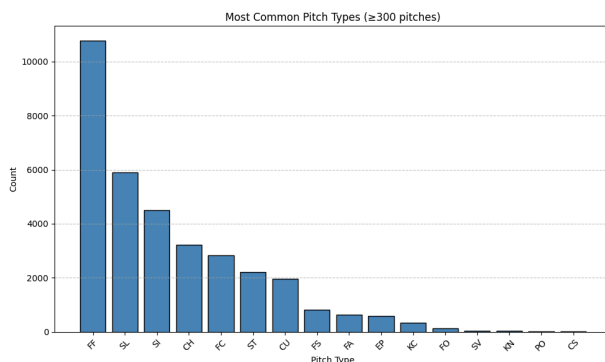


Figure 1

When we segmented pitch usage by base state and count, we noticed contextual variation: certain pitch types became more or less frequent depending on game situation. One context is the count, that is, how many balls and strikes there are.

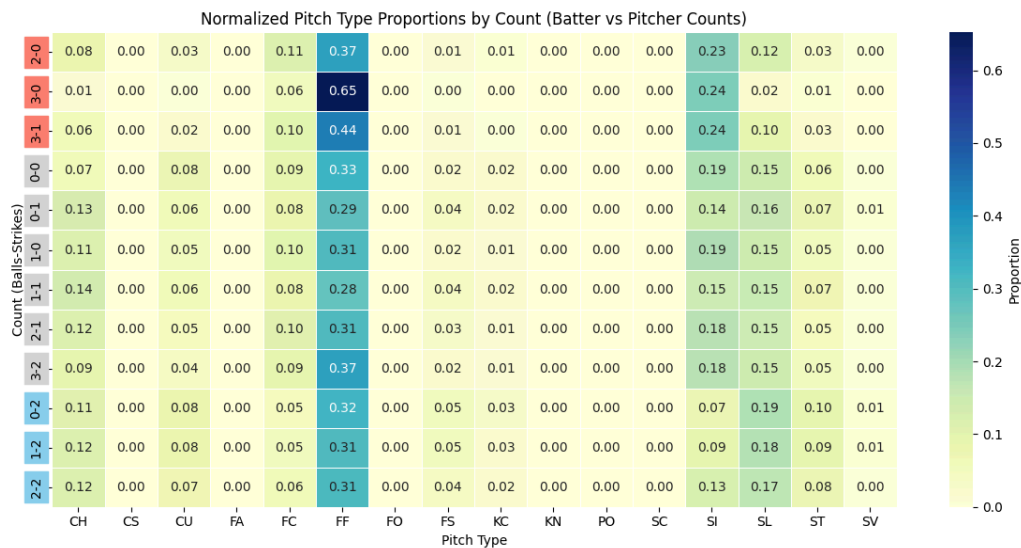


Figure 2

This heat map reflects what happens during what is generally referred to as a “pitcher’s count,” “batter’s count,” and “neutral count.” Batter’s counts, such as 2-0, 3-0, and 3-1, favor the hitter, as pitchers are more likely to throw a strike, lest they get further behind in the count and give up a walk. For most pitchers, that means they’ll rely on their most confident pitch in that situation, which we can say is their most common pitch:

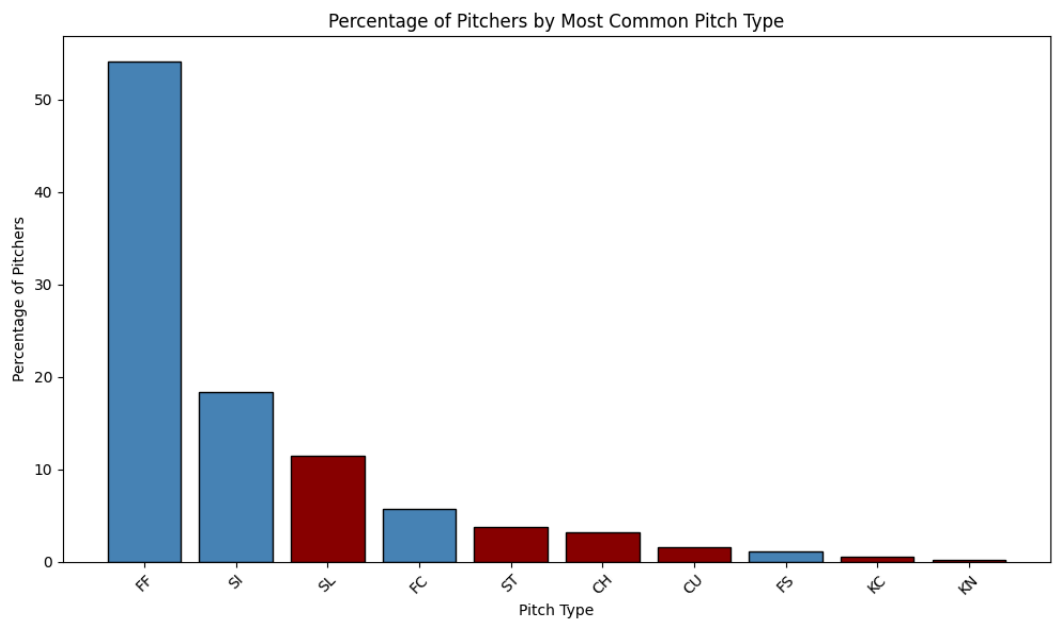


Figure 3

For most pitchers, that's the fastball, so it makes sense in a 3-0 count that a Four-Seam Fastball is thrown about 65% of the time. Pitcher's counts, like 0-2 and 1-2, put the pitcher in control, allowing them to "waste" a pitch or throw offspeed to try and bait the batter into chasing, and we can observe higher levels for offspeed pitches in such a situation. Neutral counts — including 0-0, 1-1, and 2-2 — are more evenly matched, with neither side holding a clear strategic edge.

Another context is how many runners are on base while also considering how many outs there are:

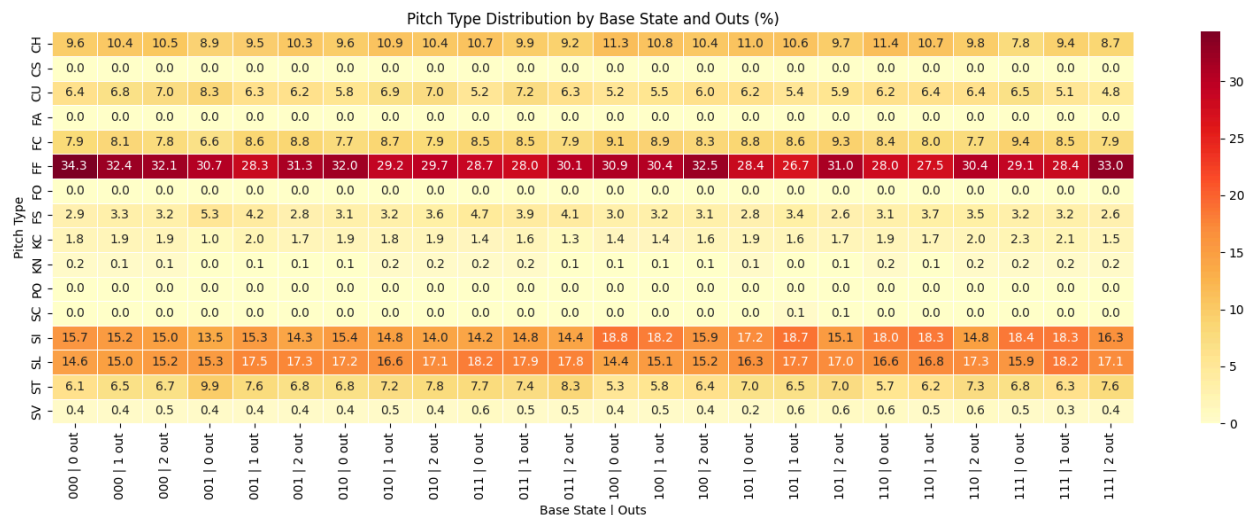


Figure 4

There are many narratives that can be derived from this heatmap, but one of the most important contextual points that isn't explicitly reflected in the data is that a pitcher will often try to achieve a **double play**, which is when he (his defense) gets the runner out at first and second base (or second and third base). From that, we can also logically deduce that pitchers, when they already have a runner on base, are more likely to set themselves up in situations where they can achieve a double play. Our heatmap helps demonstrate this: when there's a baserunner only on second base or only on third base, or baserunners on second and third base simultaneously, pitchers tend to throw more offspeed pitches (which we've established is the less common pitch for most pitchers) since there's room to 'allow' these batters obtain a walk, thereby setting up a double play opportunity. (When there are runners only on second and third base simultaneously, it isn't possible to get a routine double play.)

When there's a baserunner on first and at least one out, we observe that pitchers lean more heavily on the fastball, particularly the sinker fastball. This also makes sense logically because pitchers actually want batters to make contact, albeit only ground balls (a batted

baseball that rolls or bounces along the ground), and the sinker fastball, which has downward motion, helps accomplish that. These are just a couple of narratives that we were able to deduce, but gave us a glimpse into the complexity of pitching context.

We also observed that pitchers have highly individualized pitch preferences, which highlighted the importance of analyzing pitcher-specific tendencies rather than relying solely on general pitch distributions or game situations. For example, consider Corbin Burnes, one of the top starting pitchers in MLB. As shown in the bar graph below, Burnes heavily favors his cutter—a type of fastball—across most scenarios.

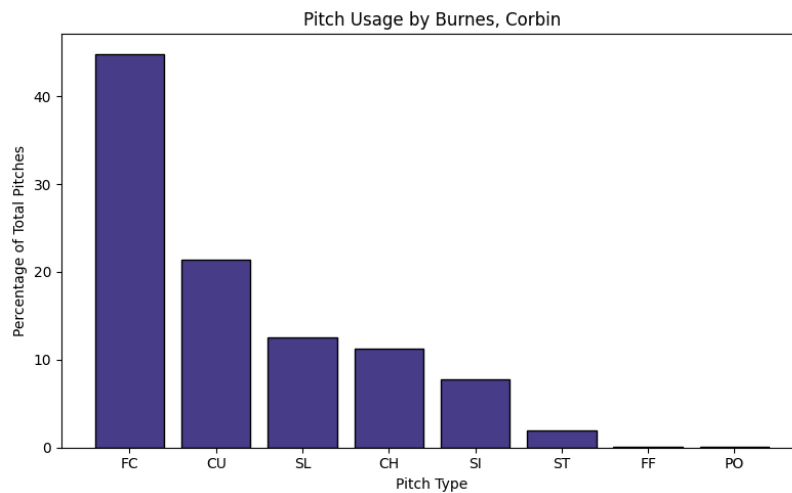


Figure 5

In contrast, another elite pitcher, Clayton Kershaw, primarily leans on his slider, an offspeed pitch, even in situations where a fastball might be expected.

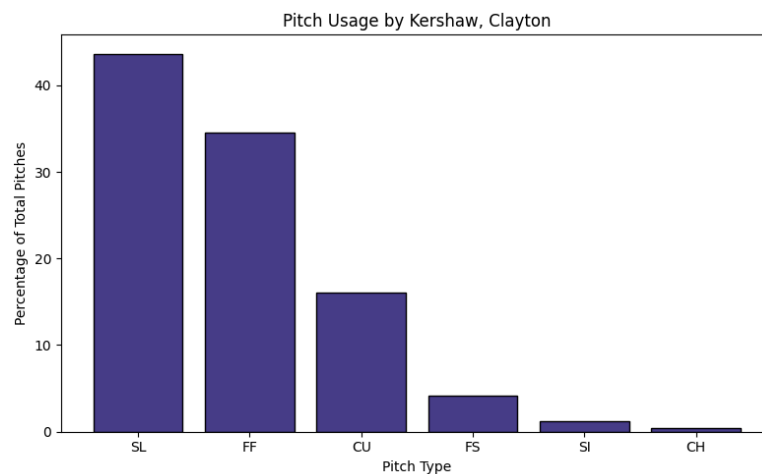


Figure 6

These differences underscore why it's critical to account for each pitcher's unique habits when building predictive models, rather than assuming uniform behavior across all players. A fastball, though the most common pitch among all pitchers, is not always the most common pitch for *individual* pitchers.

These EDA findings ultimately helped guide our feature engineering (such as adding count buckets, inning phases), and helped us choose models that were capable of capturing complex interactions and relationships.

From this, we observed several challenges:

- Class imbalance: Fastballs are thrown much more often than offspeed pitches, which can make a model biased towards always guessing a fastball. Several of the figures reflected that, and we knew from the start we would have to address this.
- Identity confounding: occurs when the same pitchers appear in both training and test sets, causing the model to memorize individual habits instead of learning generalizable patterns, leading to inflated performance (for instance, the model may think that, just because Corbin Burnes is likely to throw a cutter on a 2-2 count, other pitchers will also throw the same pitch on the same count)
- Baseball is complex, and available datasets, though rich, can never fully capture the entire context that influences a pitcher's decision. There are unpredictable factors (sudden injuries/uncomfortability, irrational loss of confidence in a certain pitch, etc.) that are simply part of pitchers being human.

To address class imbalance, we adjusted the `scale_pos_weight` hyperparameter, and to enrich the feature space, we created new variables like count buckets (categorizing ball-strike counts), inning phases (early/mid/late game), and score difference buckets.

To address identity confounding, we created two datasets: one with some overlap of pitchers between training and testing data (to model memorizing pitcher habits) and one without overlap (to test generalization to new pitchers). We used XGBoost, a tree-based machine learning method known for handling imbalanced data and complex interactions, as our main model. We also explored adjusting the classification threshold to balance fastball and offspeed prediction accuracies.

Overall, our project aims not just to predict with high accuracy but also to understand how well models can learn situational patterns versus individual pitcher habits.

Data Collection

As we explained, we believe that comparing the performance of a model that tracks identity vs. a model that doesn't will provide insight into pitchers' habits and the way they operate in certain situations. Thus, the core idea behind our project was allowing identity confounding for one version of a model, then using the same fundamental model but not allowing identity confounding. The way to do this was by manipulating the data that we fed into our models and controlling for the level of identity confounding. Thus, we created two datasets. We began the process by making a list of pitchers that we wanted to track. This list contained pitchers that threw more than 300 pitches in the year, which was intentional – it ensured that each pitcher had enough data to examine their distribution, without loss of generality (we still examined pitchers who threw not that frequently). We then, for the first dataset, filtered our large dataset of all pitches from April to September to just have the pitchers in this list. After we did that, we split the new dataset into groups by pitcher, and for each pitcher put a random 80% of their data into a train dataset and 20% into a test dataset. This is different from a record-wise split because, rather than just allowing for pitches from the same individual to enter into both train and test data, we directly enforced this to happen.

For the second dataset, we did not want to allow any pitches from the same individual into both train and test data, so we did our 80-20 split earlier: we split the original list of pitchers that threw over 300 pitches into 80% of its pitch volume and 20% of its pitch volume (meaning 80% of the pitches thrown were contained in one list, and 20% in the other...). There wasn't the same number of pitchers in each list, nor was there necessarily 80% of pitchers in one list or 20% in the other, but the pitchers in each represented 80% and 20% of the total pitches thrown in the regular season. Then we just went through our large dataset and sent data from the pitchers in the 80% of pitch volume list to a training dataset, and data from the pitchers in the 20% of pitch volume list to a test dataset. This process is called "subject-wise" splits, and it prohibits a model from tracking identity.

Since we did all of our splits here, in the preprocessing stage, we didn't do cross-validation. We needed to manually create train and test sets rather than allow numpy to randomize the sets so that we could see the effects of identity tracking and connect that with pitchers' habits.

Model Building and Accuracy Metrics

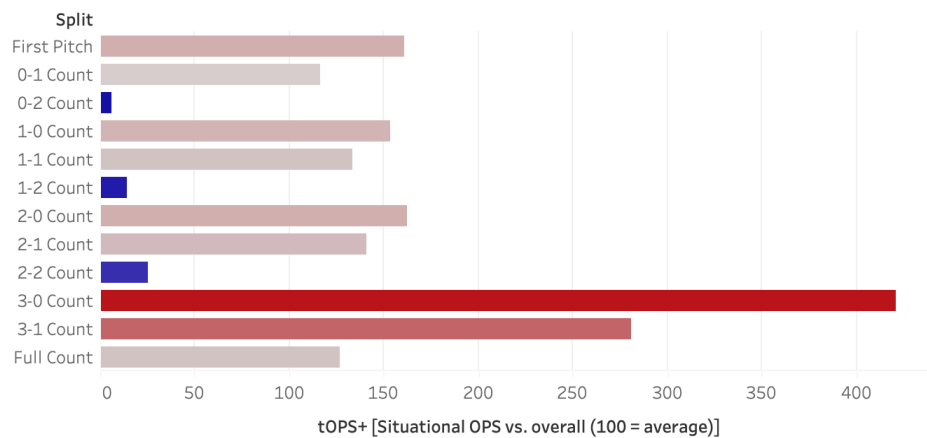
The model type we chose was XGBoost (eXtreme Gradient Boosting). It uses gradient boosted decision trees to classify pitches by sequentially improving weak learners based on previous errors. We selected this model because it performed at the highest level on our baseline test. Our baseline test consisted of only utilizing data from June, and not creating pitcher-specific models, but creating one large model. This decision allowed us to test each model type more efficiently, allowing for more model types to be tested. The model types we put through the baseline test

were Logistic Regression, K-Nearest Neighbors, Random Forest, and XGBoost. XGBoost likely performed better than the other models due to its ability to handle imbalanced data sets through its ‘weighted classification’ functions (Chen & Guestrin, 2016). As our dataset includes over 16,000 more examples of fastballs than offspeed pitches, this feature of the model was crucial to improving our predictions of offspeed pitches.

Data Preprocessing

The first step in creating the model was to preprocess the data. To begin, we engineered two new features to use in our model. The first was the score differential. We simply subtracted the score of the defending team from the score of the hitting team. This was done to make the score a relational feature, instead of just two numbers. The second feature was a ‘count bucket’. This feature groups counts into one of four buckets, based on whether the count favored the pitcher, hitter, or neither. We decided which counts would go into each bucket by comparing the tOPS+ (“leverage-based situational statistic for batters” where 100 is average) (Baseball Reference, n.d.). We chose the counts with a tOPS+ greater than 175 for hitters' counts, lower than 50 for pitchers' counts, while everything except a 3-2, or ‘full’ count, was placed in the neutral category. We decided to treat full counts as special because of their ‘do or die’ nature. Pitchers will often throw a different pitch when a strikeout or walk is imminent, compared to other ‘neutral’ situations.

tOPS+ by Count



While the other data we scraped from Baseball Savant was generally useful without any changes, certain qualitative features, such as baserunner data, needed to be converted. Baseball Savant provides baserunner data as three separate features, one for each base, with the entry either blank, signifying no runner on that particular base, or with the baserunner’s player ID if they were occupying that base. We decided not to take the specific baserunners into account because they are not predictive on their own, and including them would introduce high cardinality

categorical variables that could lead to overfitting. We decided that the presence of a runner was generally more important than the identity of the runner. Considering this, we used one-hot encoding to change the three variables to a binary format, either 1 or 0. A 1 was placed if there was a runner on that base, and a 0 if not. We used ordinal encoding to encode the other categorical features, such as the pitcher and batter's handedness (right or left), and our new count bucket feature. We decided that this technique was better suited for these features because these are multi-class categorical variables where the number of categories was small. It was also more efficient than one-hot encoding, which expands each into multiple columns. Using one-hot for too many features would have increased feature dimensionality, further decreasing the model's efficiency (Micheal, 2023).

Model Construction

As discussed previously, a core principle of our model was to use identity confounding to our advantage. As pitchers across MLB each have different mixes of pitches and different tendencies to throw certain pitches in certain circumstances, we thought that we could use this to our advantage to improve the accuracy of our model. To predict pitch type based on each pitcher, we created a loop that made a separate model for each pitcher in the dataset. The dataset was split via an 80/20 train-test split that we performed before inputting the data into the model. Because a separate model was created for each pitcher, of their total number of pitches in the overall dataset, we randomly selected 80% for the training set and 20% for the test set.

Once we had the training and test data and all the features prepared, we could run the model. When we first ran the model, we noticed that the model was predicting fastballs very accurately, but offspeed pitches very poorly, signifying that the model was overpredicting fastballs. This is where the hyperparameters became useful. We were able to tune the hyperparameter `scale_pos_weight` to lower the number of fastball predictions and increase the number of offspeed predictions. We then went about tuning the other two hyperparameters: `n_estimators` and `learning_rate`. We used a manual loop to find the optimal combination of the three hyperparameters that would not only maximize the overall accuracy of the model, but also balance the fastball and offspeed predictions, so as not to favor one class over the other. After this test, we settled on a weight multiplier for `scale_pos_weight` of 0.7, meaning that it reduces the default class imbalance adjustment by 30%. The optimal learning rate was 0.01, and the optimal number of estimators was 150. Even though the arrangement of hyperparameters didn't result in the highest overall accuracy, we opted for this combination because we valued the reduction of false positives (predicted fastball, actually offspeed). These false-positive situations are especially difficult for hitters, as if a hitter is expecting a fastball, they will generally be fooled by an off-speed pitch. On the other hand, if a hitter is expecting an offspeed pitch, it is significantly easier to adjust to the fastball and foul off the pitch, extending the at-bat.

While the first model was generally successful, we wanted to see if trends remained when we eliminated identity confounding. For the dataset for this second model, we randomly split pitchers based on the number of pitches they threw so that 80% of pitches thrown would be in the training and 20% in the test. To create a model without identity confounding, we followed the same preprocessing methods, but then for the modeling, we removed the looping mechanism and instead ran only one model, compared to upwards of 200. Similar to the first model, we ran a manual loop to test all the possible combinations of the hyperparameters, the multiplier for `scale_pos_weight`, the number of estimators, and the learning rate. For this model, we settled on values of 0.7, 200, and 0.05, respectively.

Accuracy Metrics and Results

The accuracy metric we used to evaluate these models was a confusion matrix-based evaluation. The three metrics we calculated were the accuracy ($TP+TN/\text{total}$), sensitivity ($TP/TP+FN$), and specificity ($TN/TN+FP$). Using these metrics, we were able to see the overall accuracy, the accuracy of fastball predictions, and the accuracy of offspeed predictions.

The overall accuracy of the model that used identity confounding was 61.6 (82048/133269). The model's accuracy on fastballs was 62.565% (46834/74857) and 60.286% (35214/58412) on offspeed pitches. The global model had an overall accuracy of 57.631% (76088/132026), and scored 61.568% (46384/75338) on fastballs and 52.399% (29704/56688) on offspeed. From this, we can see that individual, pitcher-specific models outperformed the singular, global model in each of the three metrics. By tailoring an XGBoost model to each pitcher, we increased the overall accuracy by 3.9 pp, fastball accuracy by 1.9 pp, and greatly increased offspeed accuracy (+6.5 pp).

Confusion Matrix - Pitcher Specific Model

cm	Predicted Fastball	Predicted Offspeed
Actual Fastball	46,834	28,023
Actual Offspeed	23,198	35,214

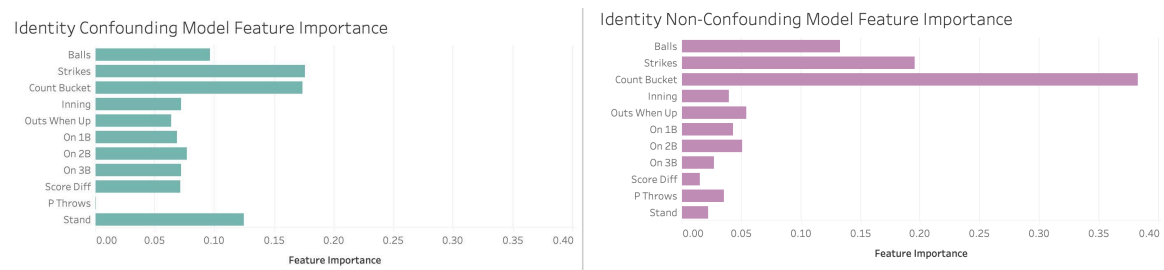
Confusion Matrix - Global Model

cm	Predicted Fastball	Predicted Offspeed
Actual Fastball	46,384	28,954
Actual Offspeed	26,984	29,704

Feature Importance

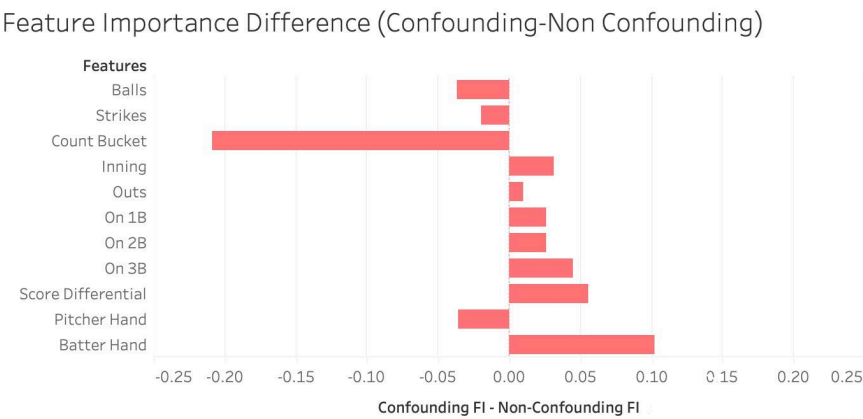
While accuracy metrics offered a broad assessment of performance, we also wanted to understand how each model made its decisions. To better comprehend what the models were

learning, we analyzed the feature importance scores generated by each model. These scores quantify how much each feature contributed to the model's decisions across all trees. In both the pitcher-specific and global models, the most important features were clearly the numbers of balls and strikes, along with the count bucket. This was intuitive because pitch type selection highly depends on the count, especially in counts closer to walks or strikeouts.



Something we found intriguing was that the global model focused intensely on just a few features, specifically count bucket, balls, and strikes, accounting for over 70% of the total importance weight. Conversely, the pitcher-specific model distributed weight more evenly, with no feature in the model having over 18% weight, suggesting that these individualized models captured a broader range of contextual factors. This likely reflects the fact that the global model, which lacks individual pitch tendencies, relied more on general patterns across all pitchers, because it consistently influenced pitch selection league-wide.

To compare how the two models prioritized different features, we plotted the difference in feature importance scores between the pitcher-specific and global models. Positive values indicate the feature had more importance in the pitcher-specific models, while a negative value reflects features more heavily emphasized in the global model. The most striking difference was in the importance of the count bucket, which the global model valued at approximately 21 pp more than the pitcher-specific model. This reinforces the idea that the global model relied heavily on general league-wide tendencies, as we would expect.



Another Interesting Result

To reiterate, the aim of our project goes beyond building a model that can predict pitches – we want to use the differences between our model that we trained to track identity and our model that was not trained to track identity to make conclusions about pitcher's habits. So, after building a model and optimizing accuracy, our project was really only halfway done: we still had yet to figure out how we could use our findings to analyze something deeper. We explored various differences between the two models, mostly with respect to optimal parameter values, but these metrics were often too interwoven with complex data science principles to reveal anything about real-world pitching trends (they revealed more about model building itself). We wanted to make a conclusion about sports.

What ended up really catching our attention was when we studied the situations in which these two models were most likely to make a mistake. To deduce this metric, we simply tracked the value of each non-target attribute for test observations that the model misspecified, and reported the most frequent situation in which the model predicted wrong. We ended up finding that our model that tracked identity had the most trouble in the first pitch of at-bats (its error rate in some of these situations was around 72-80%), while our model that did not track identity had the most trouble in pitcher's count scenarios. Its error rate in some of these situations was around 64-68%. We found this interesting, and decided to focus our efforts on deciphering why this may be.

Probability Distributions

We hypothesized that this difference had to do with a difference in individual pitchers' pitch-probability distributions and the general pitch distribution. Let's dive into this idea of pitch-probability distributions further, just to be clear.

Each pitcher has a pitch-probability distribution. That is, for the 15 pitch types in our dataset, each pitcher throws each pitch with a certain probability in a situation, and these probabilities form a probability distribution (with 15 tick marks on the x axis, if we are talking in terms of a visual). Presumably, the model that tracks identity forms in its memory a distribution for each pitcher (or at least something similar to this) because remembering pitch probabilities by identity is an easier task than remembering pitch identities by game situation. Even if this isn't exactly the case, some logic of this form is embedded in the identity tracking model to a greater extent than in the non-identity tracking model.

A general distribution exists as well as individual-based distributions. That is, for the 15 pitch types in our dataset, each pitch is thrown a certain amount of times throughout the course of the season, so every time a pitch is thrown each pitch type has a certain probability (the number of times it is thrown/the total amount of pitches thrown). These probabilities form a general probability distribution (not taking into consideration pitcher-by-pitcher differences).

So, we wanted to explore the differences in pitcher-specific probability distributions and the general pitch distribution, because we figured these differences were causing our models to perform differently. To be more specific, we hypothesized that our model that was making guesses based on pitcher-specific distributions would have most trouble in situations where pitchers' individual distributions were most different from their average individual distributions, while our model that was making guesses based on the general distribution of pitches would have most trouble in situations where the general distribution strays the most from its average self. And, if this was true, since our identity tracking model does worse on first pitches, and our non identity tracking model does worse on pitcher's counts, we would have evidence that differences exist between individual pitchers' behaviors and the general trend of pitches thrown in different scenarios (specifically first pitch and pitcher's count scenarios), which would give us insight into pitcher's habits.

So, we set out to find each pitcher's individual probability distribution in various situations, and also the general pitch distribution in various situations. To do this for the individual pitchers, we simply looped through the dataset containing pitch data from pitchers who threw more than 300 pitches in the regular season and grouped the data by pitcher, and then for each pitcher we counted how many times they threw each pitch type and divided everything by the amount of pitches they threw. The general distribution was easier to obtain: simply count how many times each pitch was thrown over the course of the season and divide by the number of pitches thrown over the course of the season.

Then, we wanted to get an understanding for the variation of these distributions. To be clear: a lot of variation would mean that a pitcher spreads themselves out and throws lots of different pitch types, and little variation means a pitcher throws a lot of the same pitch type. We started with the individual pitcher's distributions. To find the pitch variation for each pitcher was simply a variance calculation of their probability distribution, but it is key for a reader to understand the interpretation of this variance (this will be used in a later section), which was irregular and slightly unintuitive.

As we know, a variance calculation tells you how much numbers usually stray from their mean, so the variance of a distribution can often indicate how "spread out" it is. However, we must forget that definition of variance for our purposes. Let us explain why. The sum of the probabilities in the distribution for each of our pitchers is 1, obviously. So, the mean of these distributions is $1/15$ (because there are 15 pitch types in our data). Now, think about this: imagine each pitch had exactly a $1/15$ chance of being pitched in a given moment. This distribution would have a variance of 0, because each value - the mean = 0. But this distribution actually has the most amount of variation possible: each pitch type is thrown exactly the same amount (this pitcher "spreads themselves out" the most they can). Thus, from this we conclude that a lower variance actually means more variation.

Deriving the Distributions

To make conclusions about individual pitcher's distributions in first pitch situations vs. pitcher's count situations, we calculated three variances: the variance of the pitchers' distributions in an average scenario (in general), the variance of the pitchers' distributions in the first pitches of at-bats, and the variance of pitchers' distributions in pitcher's count scenarios. Then, to get a single metric to compare, we calculated the average of these variances across pitchers to get the average variations of pitchers' distributions in these different situations. So now we had three averages. (Finding the variation of each pitcher and then taking the average is not the same as finding the variation of pitchers in general, because the initial step of finding the probability distributions by pitcher makes it so that we are accounting for identity. So taking the average does not turn this into the general distribution – it is a valid way to find a single metric for a situation while still considering pitch distributions by identity).

To check our hypothesis, we simply compared our variances. Since our identity tracking model struggled the most on first pitches of at-bats, we expected and wanted to find that there was a big difference between the variation of pitchers (the pitchers' distributions) on first pitches vs. in a general/average situation. And we found exactly that: pitchers' distributions tended to have a variance of 0.030452 on first pitch situations, while in a general/average situation they tended to have a variance of .023449. In other words, pitchers tended to vary their pitches 29.9% less in first pitch situations than in a general situation. The important part here isn't that they vary less or more, but that they vary 29.9% *differently*: this explains why the model that tracks/follows pitchers' distributions performed most poorly in this scenario. In pitcher's counts, pitchers' distributions were more similar to the general distribution as they varied only 22.2% differently: still a lot, but not as much, which is why the model had more trouble with first pitches rather than pitcher's counts.

From this, we concluded that pitchers tend to act differently than they usually do in first pitch situations. This is valuable information on its own, but to make an even stronger conclusion, we wanted to examine our model that did not track identity so that we could discover properties of the general distribution of pitches and compare them to the individual pitchers' distributions.

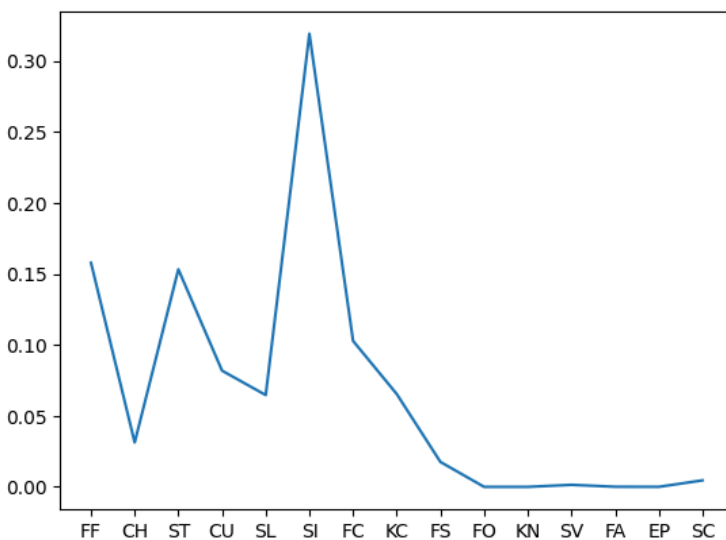
To make conclusions about the general pitch distribution in first pitch situations vs. pitcher's count situations, we again calculated three variances: the variance of the general distribution in an average scenario (in general), the variance of the general distribution in the first pitches of at-bats, and the variance of the general distribution in pitcher's count scenarios. This task was simpler, since there is only one general distribution and thus already only one metric to compare (we didn't have to take averages like the last time).

To check our hypothesis about the general distribution, we simply compared our variances again. We expected and hoped to find that there was a big difference between the general distribution on pitcher's counts vs. the overall general distribution in an average scenario. We found that the variance of the general distribution on pitcher's counts was .006655355, while the variance of the general distribution on average was .00737537. On first pitch scenarios, the variance of the general distribution was .00800737. Hence, just as we had hoped, we found that pitchers in general tend to vary their pitches 10% more on pitcher's count situations than on average, while on first pitch scenarios they only varied 8% differently, and there was less variation (this will factor into our final conclusion). This explains why our non-identity tracking model performed poorly in pitcher's count scenarios, and gives us insight into the general distribution of pitches: there is much more variation in pitch types thrown, in a general sense, on pitcher's counts than other situations.

Our Final, Extended Analysis

To bring this story to a satisfying conclusion, we wanted to present an intuitive (less number-related) idea we had of what we can decipher from the difference between individual pitchers' distributions and the general pitch distribution.

For the general pitch distribution to look the way it does*, some pitchers have to throw the more obscure pitches more often. So, what we hypothesize from all of our analysis above is that pitcher's throw their own kind of pitches on pitcher's counts, which may be more obscure in the general distribution (hence the high error rate in these situations for our non-identity tracking model), and throw a kind of regimented set of pitches on first pitch counts which may be less drawn from their own distribution (hence the high error rate in these situations for our identity confounding model) and more generally mandated staples of first pitch situations.



*general distribution of pitches

As we saw from one of the very first figures in the introduction, certain pitchers do definitely have much different distributions of pitches. And, looking also at a quick list of facts from our earlier analysis –

- Individual pitchers throw most differently than their usual selves in first pitch situations
 - This difference is actually that they throw with less variation in these situations (a higher variance) – evidence of throwing a “regimented” set of “accepted” early-count pitches
 - Pitchers in general throw most differently from the general distribution of pitchers in pitcher’s count scenarios
 - This difference is that they throw with more variation than the usual general distribution – evidence that they are tapping into more their more personal pitches, pitches that may be obscure in the general distribution
- our hypothesis doesn’t require much more analysis to understand.

References:

Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system.

<https://doi.org/10.48550/ARXIV.1603.02754>

Dhar, A. (2022, December 16). Predicting pitches. Bruin Sports Analytics.

<https://www.bruinsportsanalytics.com/post/predicting-pitches>

Lee, J. S. (2022). Prediction of pitch type and location in baseball using ensemble model of deep neural networks. Journal of Sports Analytics, 8(2), 115–126. <https://doi.org/10.3233/JSA-200559>

Micheal, O. (2023, August 27). A comprehensive comparison between one-hot and ordinal encoding. Medium.

<https://medium.com/@oyebamijimicheal10/a-comprehensive-comparison-between-one-hot-and-ordinal-encoding-6f899c4f08b3>

tOPS+—Baseball Bullpen. (n.d.). Baseball Reference.

<https://www.baseball-reference.com/bullpen/TOPS+>

Voros, M. (2024, July 15). Handling imbalanced datasets with xgboost: Optimizing model performance with smart parameter tuning. Medium.

<https://medium.com/@mate.voros1998/handling-imbalanced-datasets-with-xgboost-optimizing-model-performance-with-smart-parameter-tuning-18568c7783cf>